

Queues Crash Course

Apache Kafka



What is Apache Kafka?

- Distributed streaming platform
- For creation, storage, and processing of real-time data streams
- Developed by LinkedIn and donated to Apache Software Foundation
- Core infrastructure technology for data streams in real time

The purpose of Kafka

- **Publish/Subscribe:** Kafka acts as a message broker, allowing producers to publish data and consumers to subscribe to the data streams.
- **Storage:** Kafka provides durable storage for streams of records. It can store data with configurable retention policies, ensuring data availability for a specified period.
- **Data Processing:** Kafka Streams and Kafka Connect frameworks allow for real-time stream processing and integration with various data sources and sinks.

Kafka usecases

- **Real-Time Analytics**

Process and analyze data in real-time, gaining insights that drive business decisions.

Kafka usecases

- Real-Time Analytics
- **Event Sourcing**

Kafka's ability to handle high volumes of event data makes it ideal for event sourcing architectures, where state changes are captured as a series of immutable events.

Kafka usecases

- Real-Time Analytics
- Event Sourcing
- **Log Aggregation**

Kafka aggregates log files from multiple servers and systems, centralizing the log data for further processing and analysis.

Kafka usecases

- Real-Time Analytics
- Event Sourcing
- Log Aggregation
- **Metrics Collection**

Kafka collects and stores metrics data, enabling monitoring and alerting systems to function effectively.

Kafka usecases

- Real-Time Analytics
- Event Sourcing
- Log Aggregation
- Metrics Collection
- **Data Integration**

Kafka integrates data across various systems, acting as a bridge between different data platforms and ensuring seamless data flow.

Kafka usecases

- Real-Time Analytics
- Event Sourcing
- Log Aggregation
- Metrics Collection
- Data Integration

It's design and capabilities make it an essential tool for modern data infrastructure, supporting a wide range of applications that require robust, scalable, and real-time data processing capabilities.

The history behind Kafka

Linkedin. 2008

- User activity logs, application metrics, and operational monitoring
- Existing solutions were not scalable

Initial Challenges

- **Scalability:** LinkedIn needed a system that could scale horizontally to handle the vast and ever-growing amount of data.
- **Durability:** Ensuring data integrity and reliability was critical, as data loss or corruption could have significant business implications.
- **Latency:** Real-time data processing was essential for timely analytics and operational insights.

Origin story

- Designed from the ground up to be a distributed, partitioned, and replicated log service.
- By 2010 demonstrating its effectiveness in LinkedIn
- In 2011 LinkedIn donated it to Apache Software Foundation
 - Community Collaboration
 - Wider Adoption
 - Standardization

Key contributors and milestones

- **2011:** Kafka is open-sourced under the Apache Software Foundation.
- **2012:** v0.7 is released, major improvements in performance and stability.
- **2013:** v0.8 introduces replication, enhancing durability and fault tolerance.
- **2014:** Confluent is established to provide commercial support.
- **2015:** v0.9 adds security features, SSL encryption and authentication.
- **2016:** v0.10 introduces Kafka Streams: native stream processing capabilities.
- **2017:** v1.0 is released, marking its maturity and widespread adoption.

Key contributors and milestones

- **2011:** Kafka is open-sourced under the Apache Software Foundation.
- **2012:** v0.7 is released, major improvements in performance and stability.
- **2013:** v0.8 introduces replication, enhancing durability and fault tolerance.
- **2014:** Confluent is established to provide commercial support.
- **2015:** v0.9 adds security features, SSL encryption and authentication.
- **2016:** v0.10 introduces Kafka Streams: native stream processing capabilities.
- **2017:** v1.0 is released, marking its maturity and widespread adoption.
- **2020:** Kafka introduces the KRaft mode, get rid of dep on ZooKeeper.
- **2022:** v3.0, stable support for KRaft.
- **2024:** v4.0 preview. Major milestone with KRaft fully replacing ZooKeeper.

Key contributors and milestones

- **2020:** Kafka introduces the KRaft mode, get rid of dep on ZooKeeper.
- **2022:** v3.0, stable support for KRaft.
- **2024:** v4.0 preview. Major milestone with KRaft fully replacing ZooKeeper.
- **2025:** v4.0
 - Kafka Raft (KRaft) instead of ZK
 - Tiered Storage
 - Kafka Queues

RabbitMQ:
добавляет
Streams

Redis:
добавляет
Streams

Kafka:
добавляет
Queues



Architecture of Apache Kafka

Kafka components and how they interact

- Brokers
 - Brokers are the servers that form the Kafka cluster.
 - Each broker is identified by a unique ID and is responsible for storing and managing the data for its assigned partitions.
 - Each partition has one broker acting as the leader and one or more brokers acting as followers.

Brokers

- **Data Storage:** Brokers store data in the form of logs on disk. Each log is a sequence of records that are appended to a partition.
- **Replication:** Brokers replicate data to ensure fault tolerance. If a leader broker fails, one of the followers is promoted to the leader.
- **Load Balancing:** partitions distributed across multiple brokers. The load is balanced, preventing any broker becoming a bottleneck.
- **Fault Tolerance:** Brokers monitor each other's health through heartbeats. In case of failure, remaining brokers take over.

Brokers: leader and follower

- **Leader Broker:** For each partition, one broker is designated as the leader. The leader is responsible for all read and write operations for its partition.
- **Follower Brokers:** Brokers that replicate the partition are called followers. Followers constantly replicate the data from the leader. They do not serve client requests directly but are ready to take over.
- **In-Sync Replicas (ISR):** Brokers that are fully in sync with the leader. Data is written to all ISRs to guarantee durability and fault tolerance.
- **Leader Election:** When a leader broker fails, a new leader is elected from the ISR. This election is managed either by ZooKeeper or by the KRaft (Kafka Raft).

Kafka components and how they interact

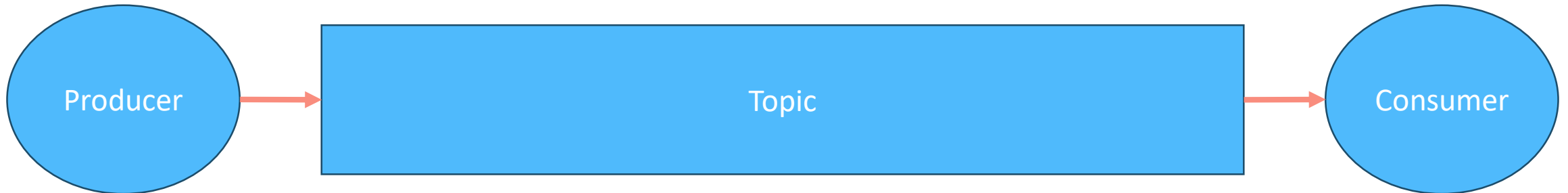
- Brokers
- Topics and Partitions
 - Topic is a media, to which records are published
 - Topics are always multi-subscriber
 - Each topic is split into partitions for parallelism and scalability
 - Partitions can be assigned based on keys

Topics

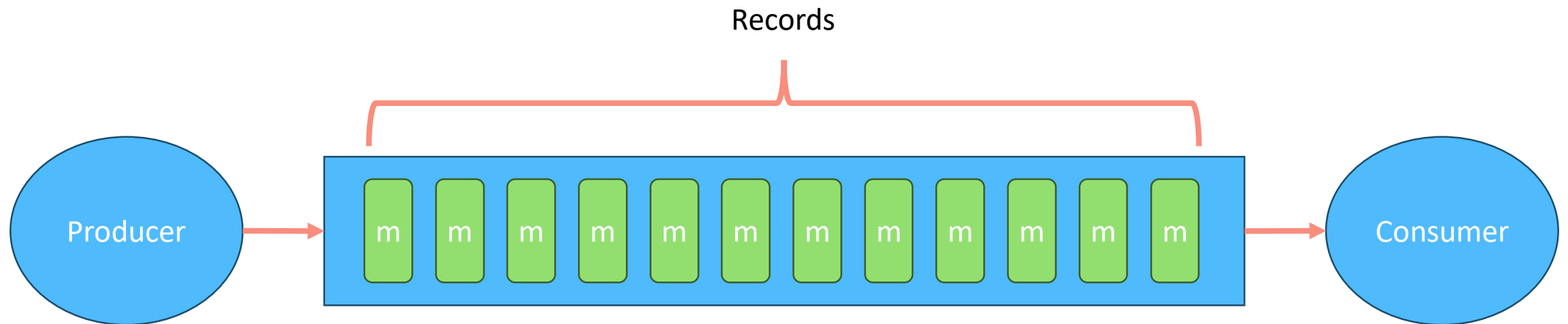
- **Topic** is a logical channel to which records are published and from which consumers can read.
- **Multi-Subscriber Model:**
Topics support multiple producers and consumers.
- **Immutable Logs:**
Records written to a topic are immutable.
- **Strict Order:**
Records are stored in the order they were produced.

Topics

- **Topic** is a logical channel to which records are published and from which consumers can read.



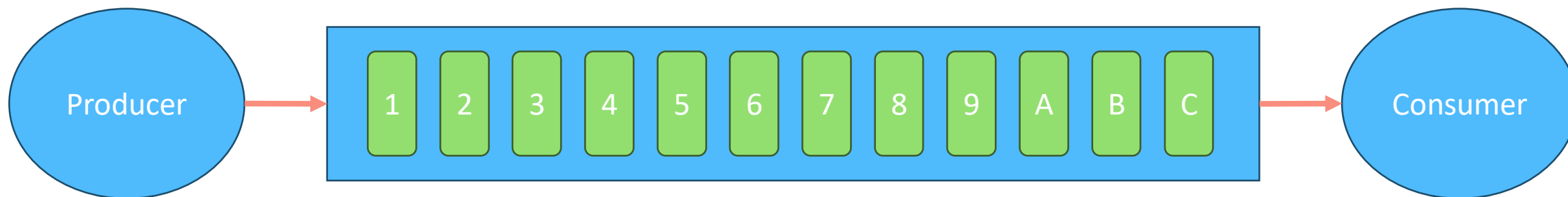
Topics



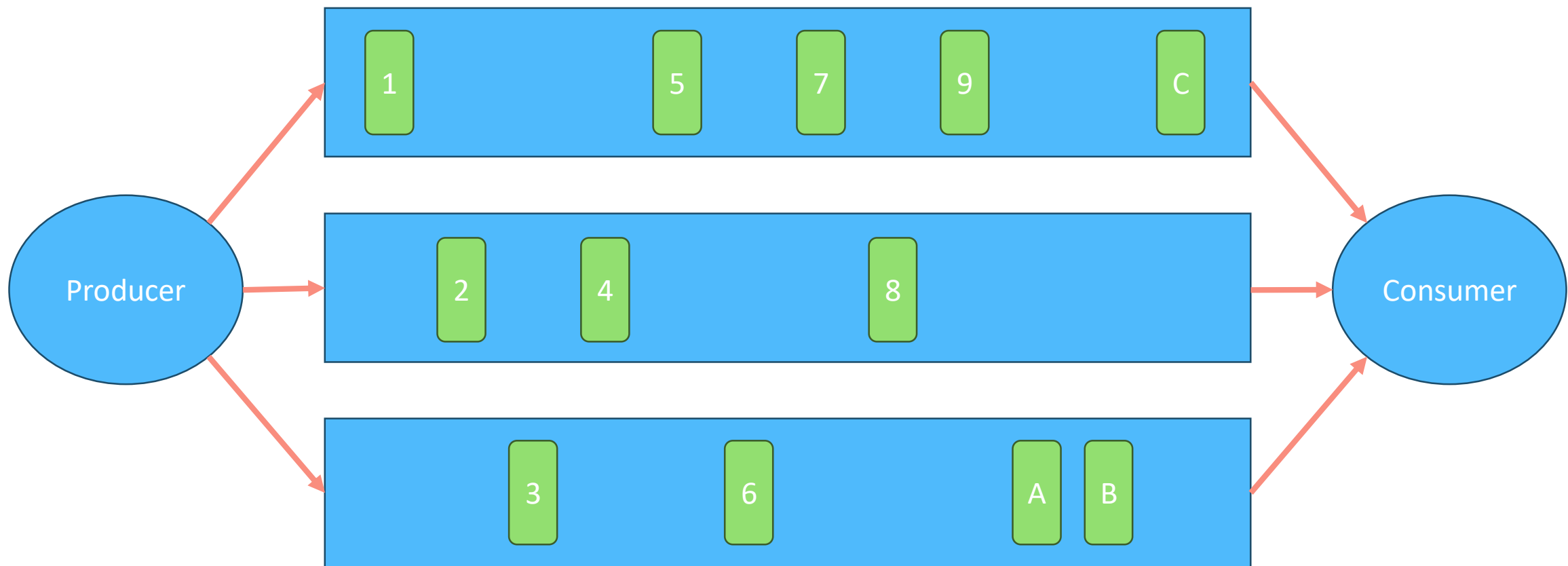
Topic partitioning

- Each topic can have one or more partitions
- Each partition is an ordered, immutable sequence of records.
- **Scalability:** Partitions enable Kafka to scale horizontally.
- **Parallelism:** Partitions allow multiple consumers to read from a topic concurrently.
- Each consumer in a consumer group can read from a different partition, enabling parallel data processing.

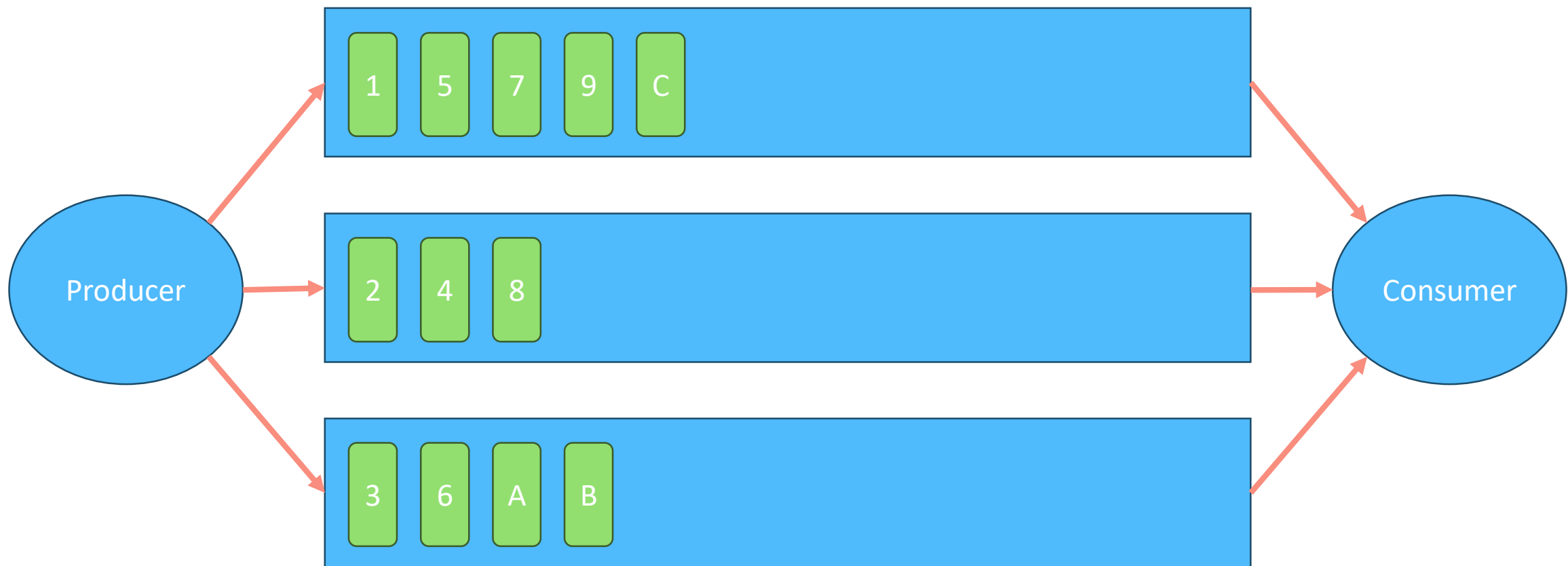
Topic partitioning



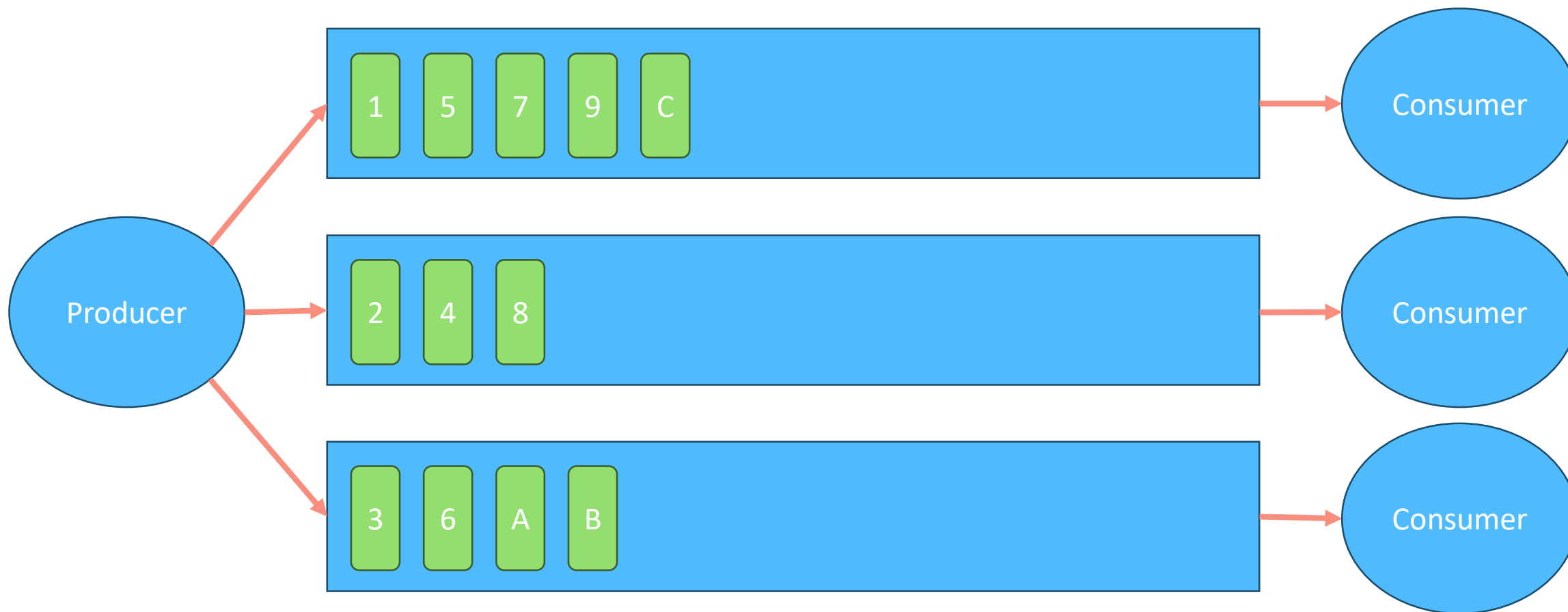
Topic partitioning



Topic partitioning



Topic partitioning



Topic Partitioning Strategy

- **DefaultPartitioner (UniformStickyPartitioner)**

Producers can specify partition

Producers can specify a key for each record.

Kafka uses this key to determine the partition.

Records with the same key are sent to the same partition.

Stickiness: producer sticks to partition.

- **Round-Robin:**

If no key is provided, records distributed in a round-robin manner.

- **Custom Partitioners:**

Kafka allows for custom partitioning logic to be implemented.

Important Partition Configuration

- **Number of Partitions:**

The number of partitions for a topic is specified on creation.

This number can be increased later but not decreased.

The number of partitions affects Kafka's performance.

More partitions can improve throughput but increase the load.

- **Replication Factor:**

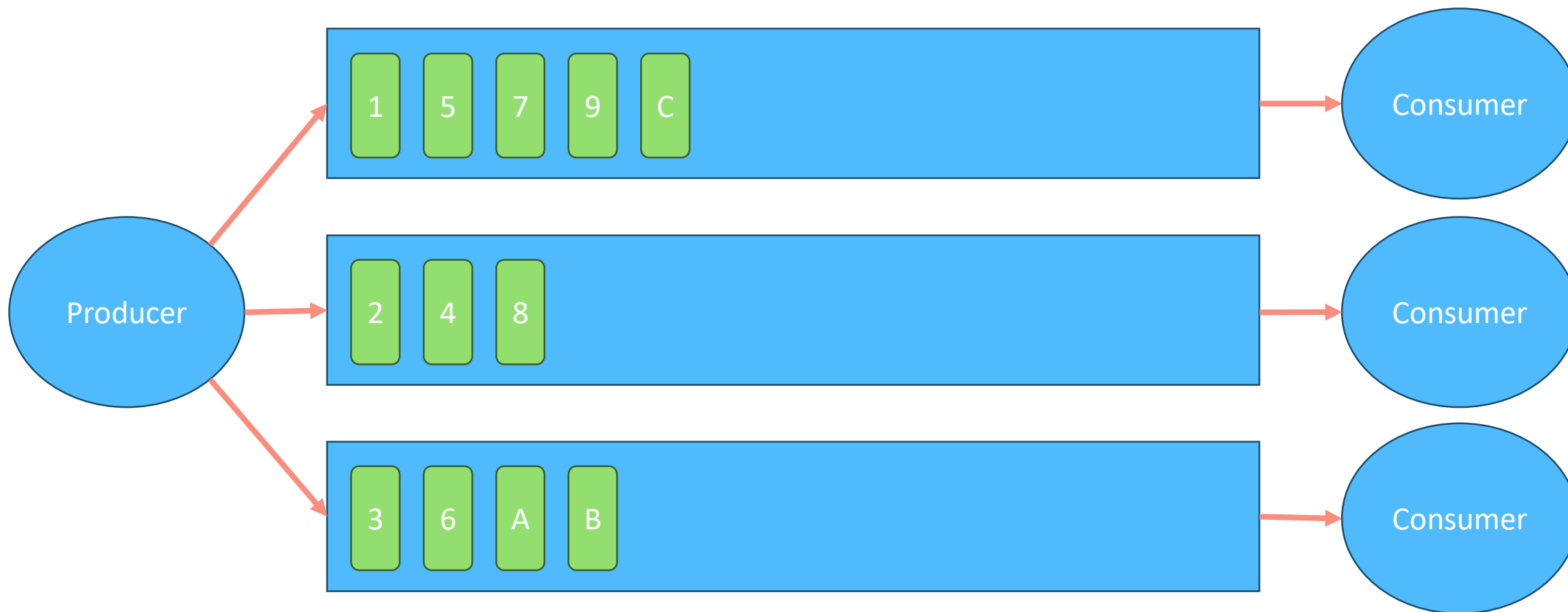
Each partition can be replicated across multiple brokers.

The replication factor is defined at the topic level.

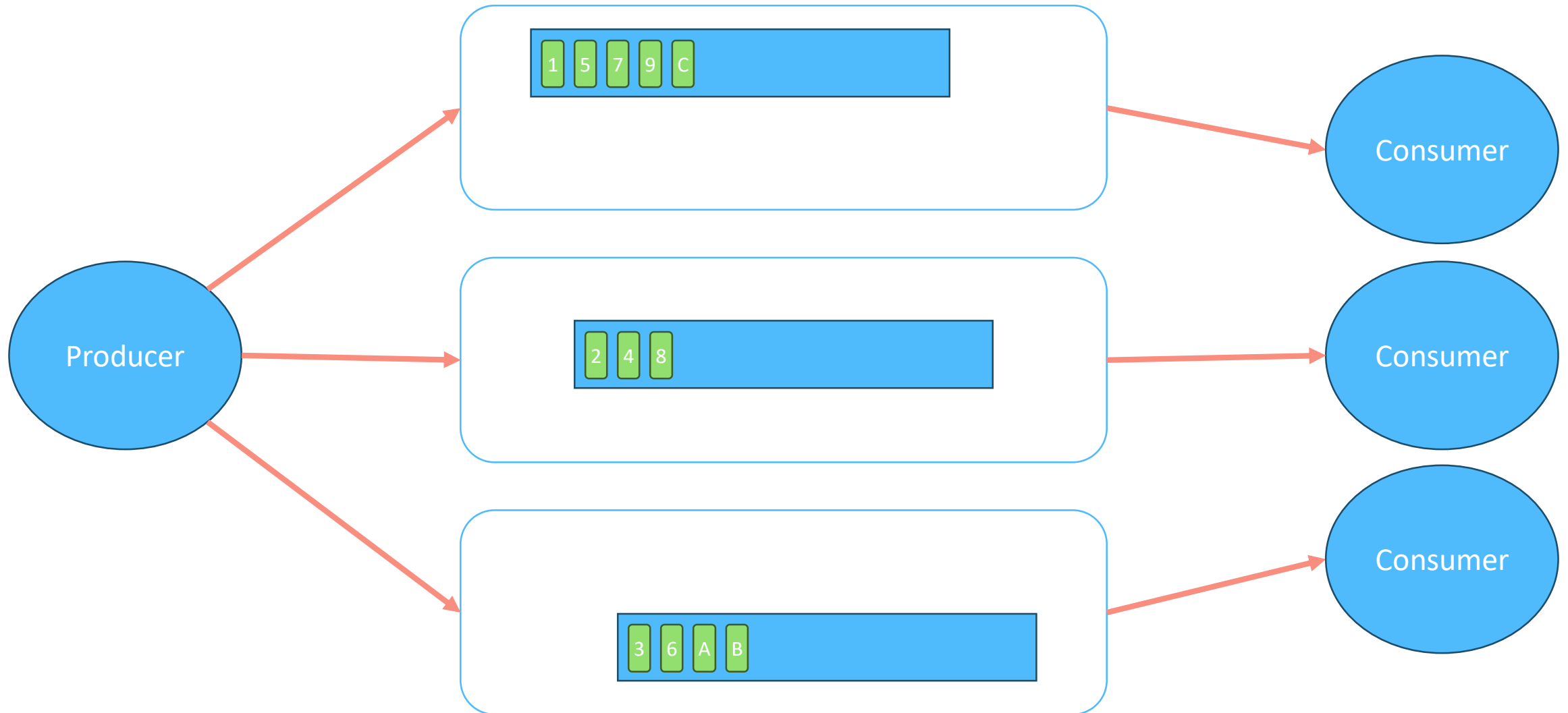
- **Retention Policies:**

Kafka allows to delete old records by time or by size

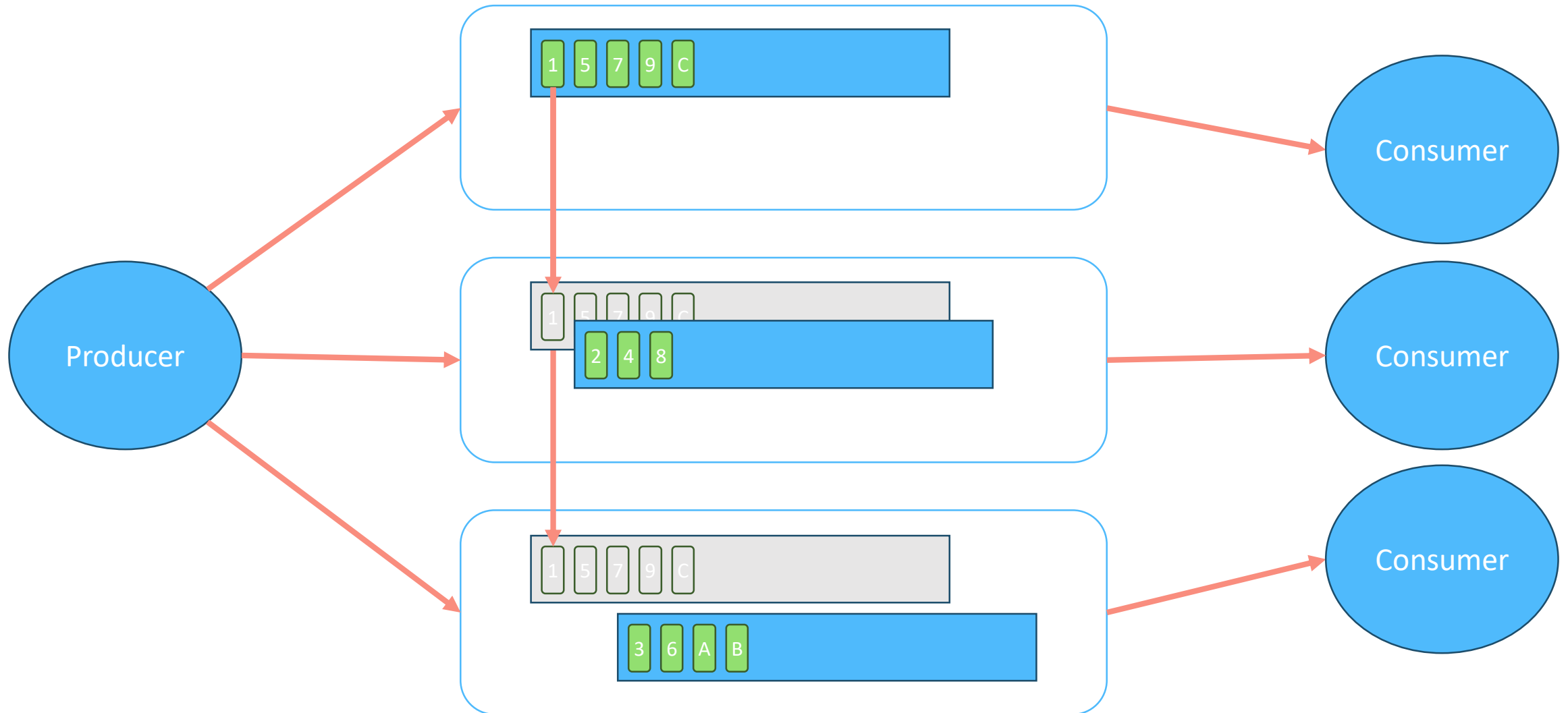
Topic partitioning



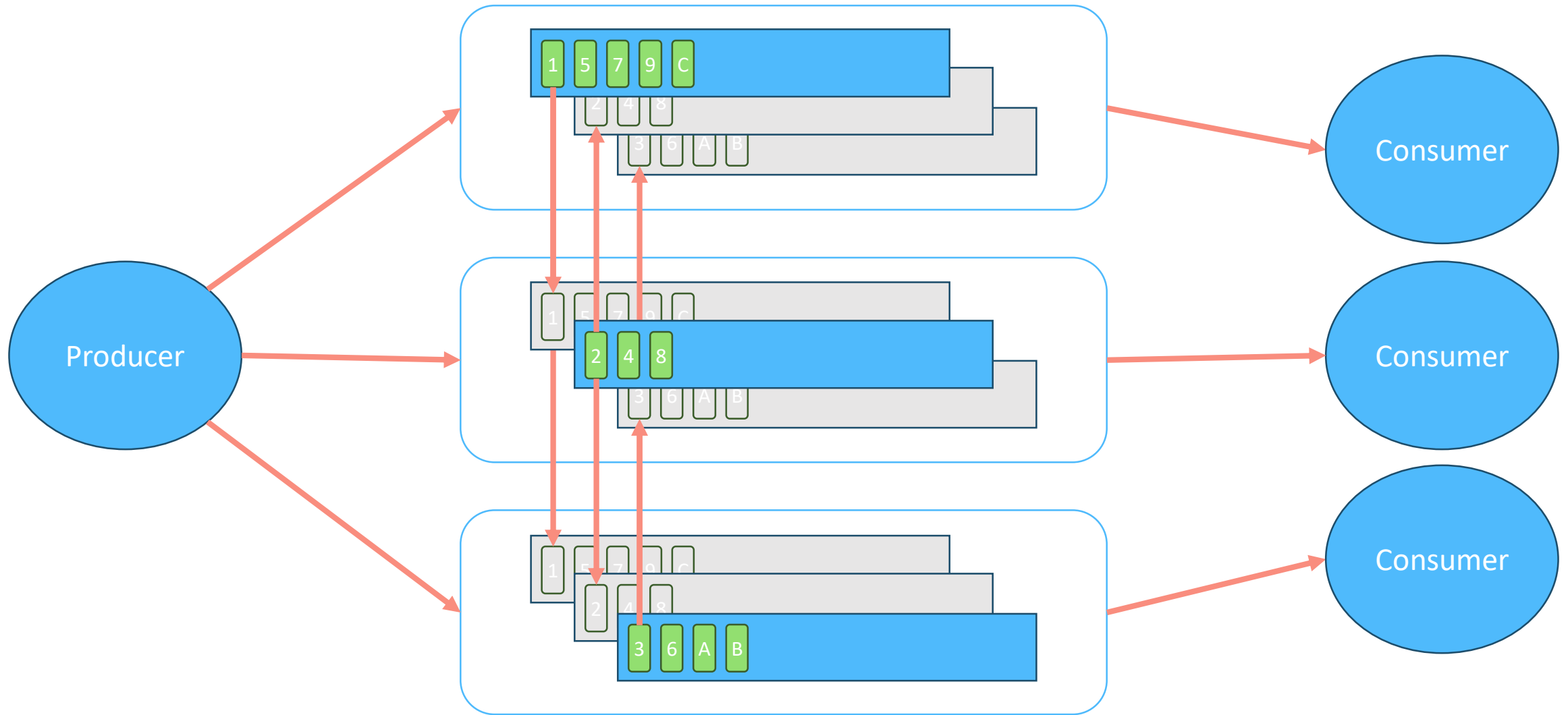
Topic partitioning and replication

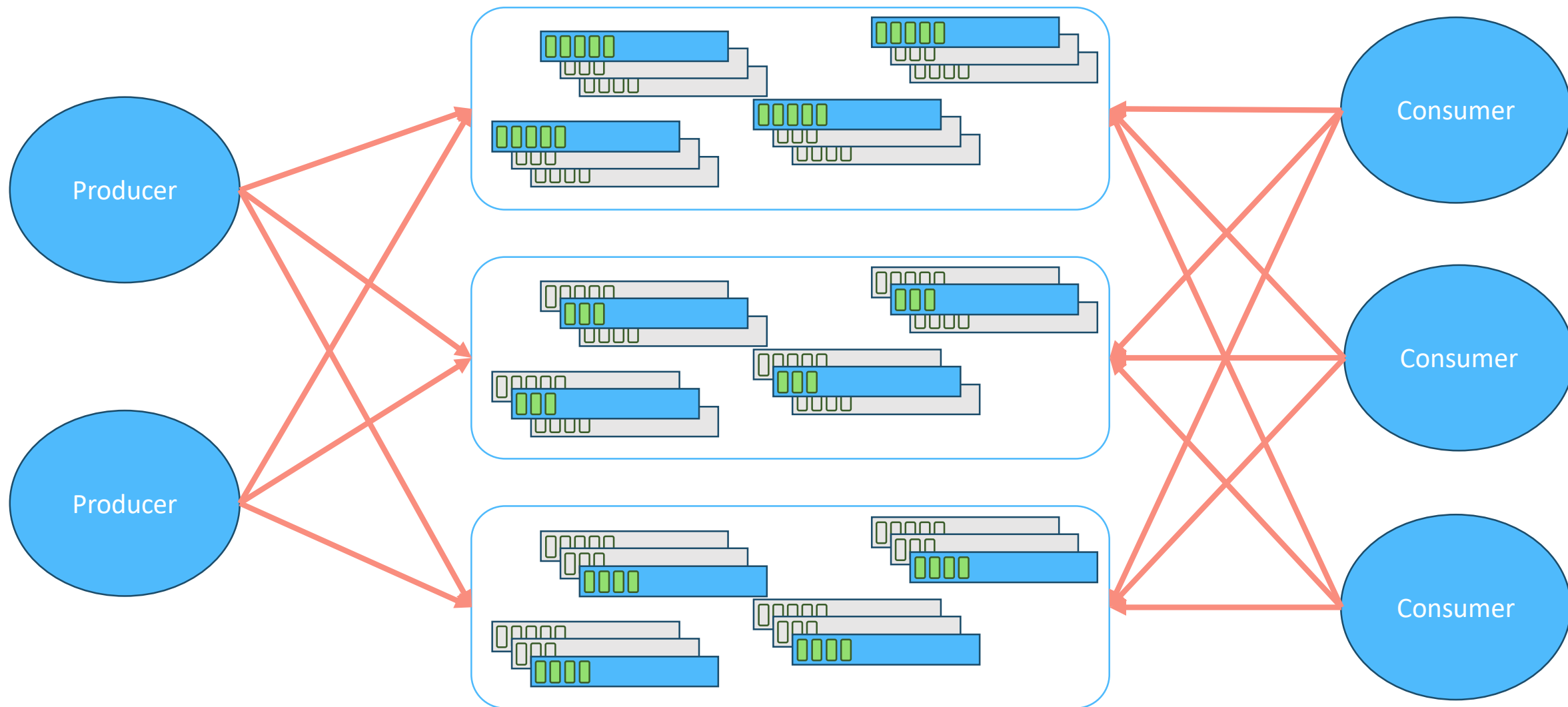


Topic partitioning and replication



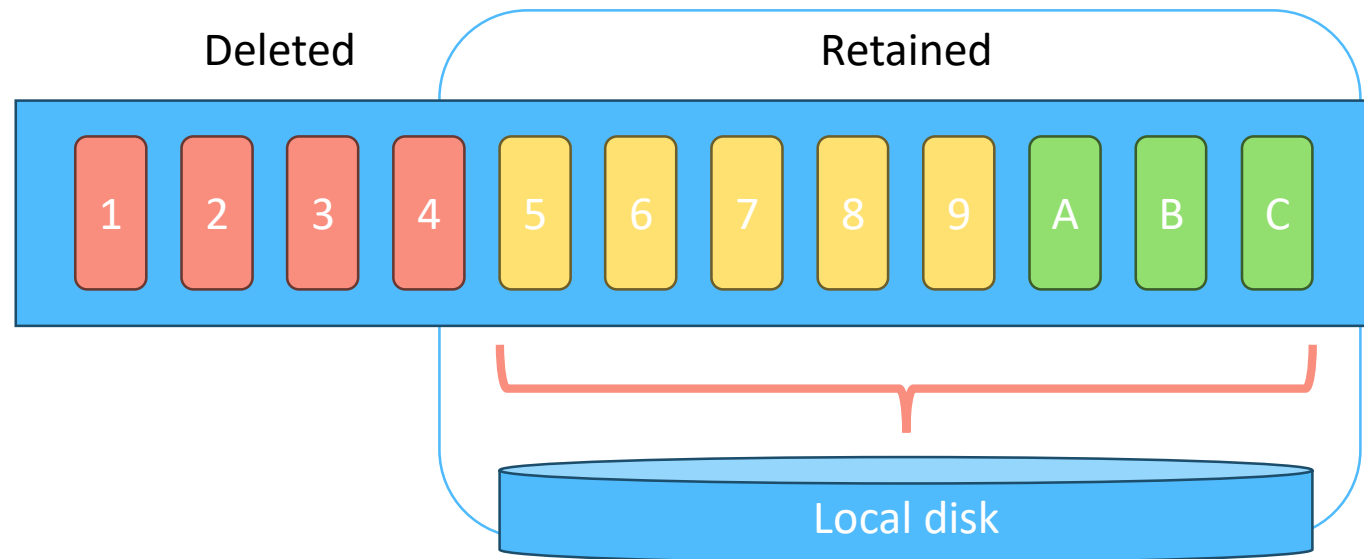
Topic partitioning and replication





Kafka Log Retention

- `log.retention.hours=<hours>` to keep
- `log.retention.bytes=<bytes>` to keep

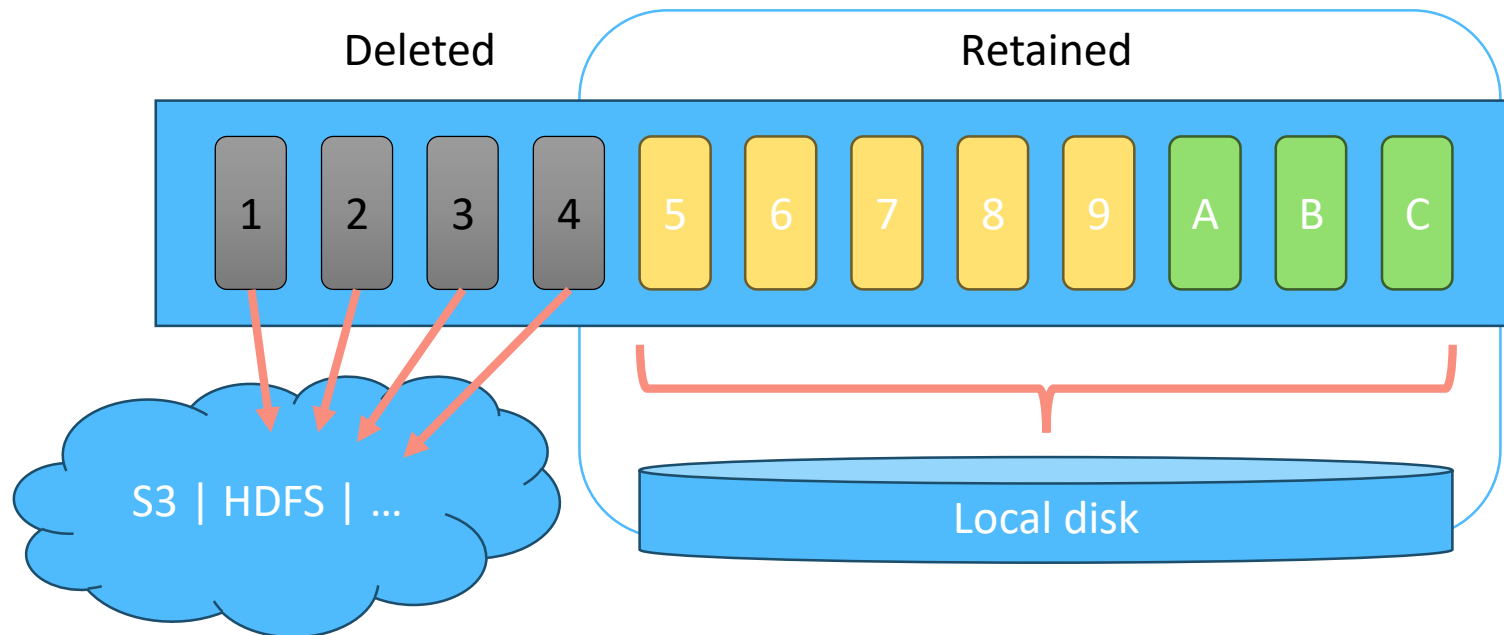


Tiered Storage in Apache Kafka 4.0

- **Extended Retention:**
Maintain historical data indefinitely without overwhelming local disks.
- **Active Data:**
Recent data remains on local disks for quick access.
- **Archived Data:**
Older segments are transferred to a remote tier transparently.

Kafka Log Retention

- `remote.log.enabled=true`



Kafka components and how they interact

- Brokers
- Topics and Partitions
- Records (or messages)
- A record consists of a key, a value, and metadata.
- Records are organized in a log
- Each record have unique offset within partition.
- Records are stored durably across the cluster.

Kafka record properties

- **Key:** An optional identifier.
Can be used for partitioning and affinity.
- **Value:** The main payload of the record.
Can be any sequence of bytes up to 1MB.
- **Timestamp:** The time of record creation or log appending.
Kafka supports two types of timestamps:
create time and log append time.
- **Offset:** A unique identifier of record within a partition.
Used by consumer to track their position in the log.
- **Headers:** Optional metadata associated with the record.
Can be used for additional context or processing instructions.

Kafka components and how they interact

- Brokers
- Topics and Partitions
- Records
- Producers
- Client applications that ***publish records*** to Kafka ***topics***
- Can choose the level of acknowledgment (acks: 0/1/all)

Kafka Producers

- Client applications that publish (write) records to Kafka topics.
- **Synchronous Send:**
The producer sends a record and waits for a response.
- **Asynchronous Send:**
The producer sends a record and does not wait for a response.

Producer acknowledgement strategies (acks).

- **acks=0:**

The producer does not wait for any acknowledgment from the broker.
The lowest latency but no guarantee of record receipt.

- **acks=1:**

The producer waits for the acknowledge from the leader broker.
This setting provides a balance between latency and durability.

- **acks=all:**

The producer waits for acks from all in-sync replicas (ISR).
Highest level of durability and fault tolerance.
The record is guaranteed to be replicated to multiple brokers.

Kafka components and how they interact

- Brokers
 - Topics and Partitions
 - Records
 - Producers
 - Consumers
- Client applications that ***subscribe*** to Kafka ***topics*** and process the ***records***.
 - Organized in groups to work on records from a set of partitions
 - Each partition in one topic is consumed by only one consumer in the group*
 - Track their position in each partition using offsets

* Except for Kafka Queues enabled topic in 4.0+

Consumers

Client applications that read (consume) records from Kafka topics.

- **Polling Model:**

Consumers use a polling model to fetch records from brokers.
They periodically request data and process the received records.

- **Offset Management:**

Consumers track their position in each partition using offsets.
Kafka provides automatic and manual offset management.

- **Consumer Groups:**

Consumers are organised in groups, that spread the consumption.
Each group consume topic independently.

Offset Management

- **Commit Strategy:**

Consumers commit offsets to Kafka to indicate their progress.

These offsets can be stored in Kafka's internal topics or external systems.

- **Automatic Commit:** Offsets are committed automatically at regular intervals.
- **Manual Commit:** Consumers manually commit offsets after processing.

- **Offset Storage:**

Kafka stores offsets in an internal topic called `__consumer_offsets`.

Consumer groups

- Each consumer in the group is responsible for consuming records from one or more partitions.
- **Partition Assignment:**
Kafka ensures that each partition in a topic is consumed by only one consumer in the group at a time.
This prevents duplicate processing and enables parallel consumption.
- **Rebalancing:**
When a consumer joins or leaves the group, Kafka triggers a rebalance to reassign partitions to consumers.
- **Heartbeat Mechanism:**
Consumers send heartbeats to the Kafka broker to indicate its liveness.

Kafka Queues (Work Queue Semantics)

- **Introduced in Kafka 4.0** to simplify queue-style messaging
- **Single Consumer per Record:**
Each message is processed by only one consumer in the group
- **Queue Topics:**
Enable with: `queue.topic.enable=true`
Automatic scaling of consumers without rebalancing
Partitioning hidden from the client
- **Virtual Partitions:**
Internally used by Kafka for scaling and distribution

Consumer rebalancing

Rebalances can happen in the following conditions:

- a consumer joins or leaves the group
- partitions are added to a topic

Rebalance strategies and approaches

- Consumer Eager Rebalancing (Stop the World)
- Consumer Cooperative Rebalance (Incremental Rebalance)
- Consumer Static Group Membership

Broker

Topic A
Partition 0

Topic A
Partition 1

Topic B
Partition 0

Topic B
Partition 1

Topic B
Partition 2

Broker 1

Topic A
Partition 0

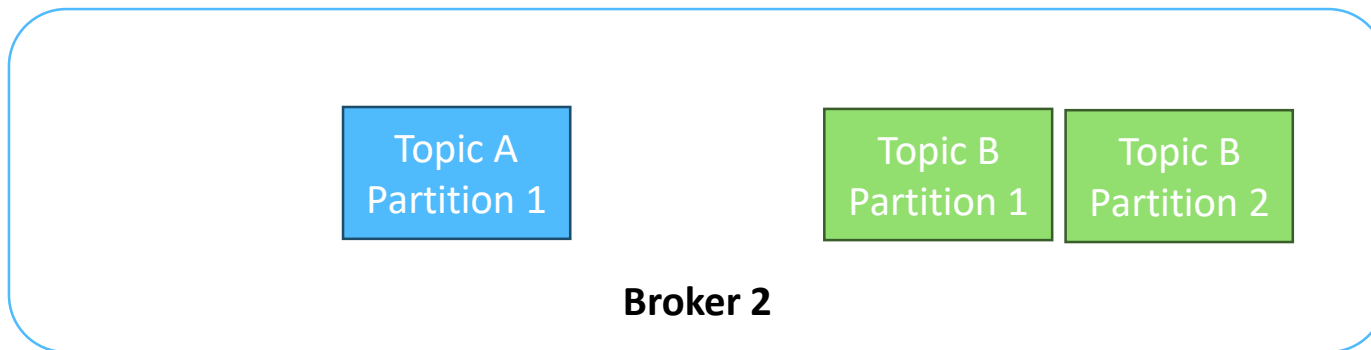
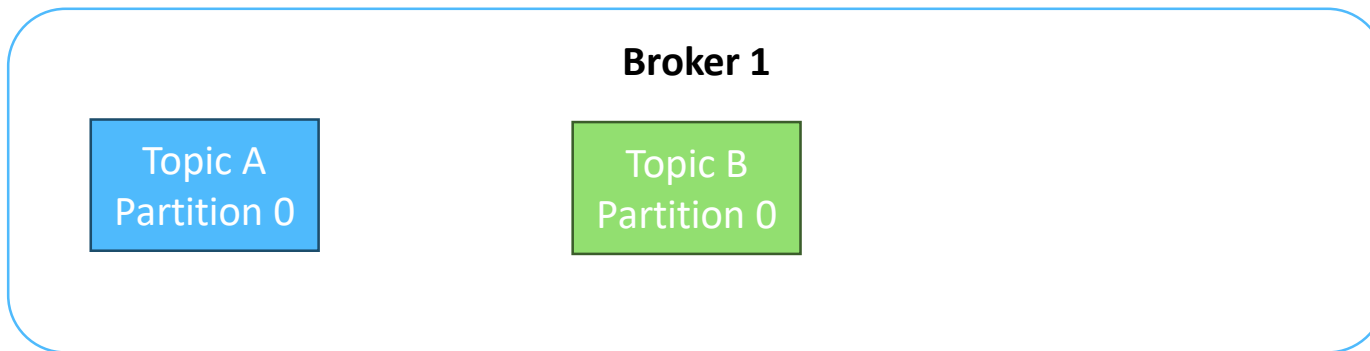
Topic A
Partition 1

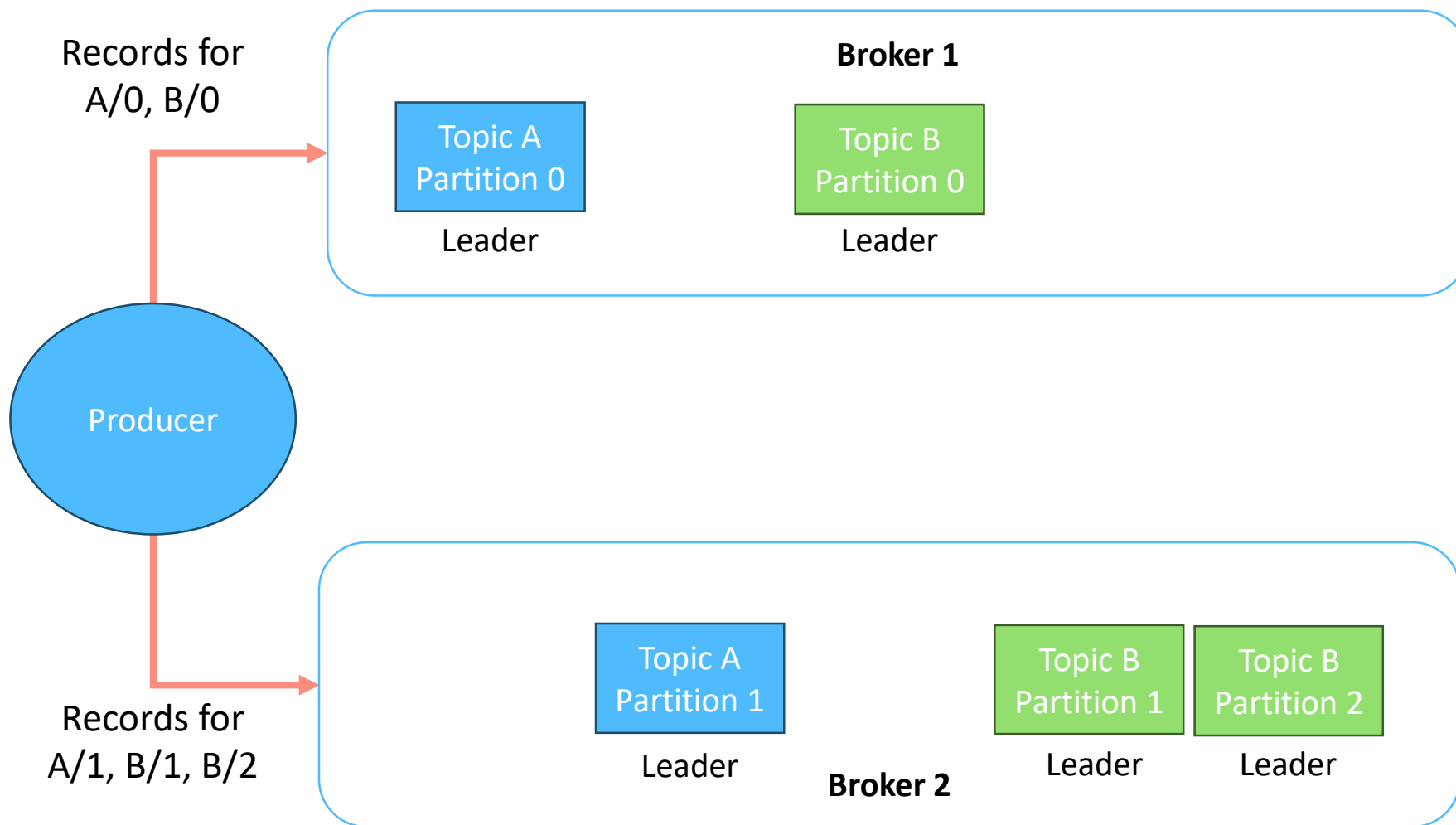
Topic B
Partition 0

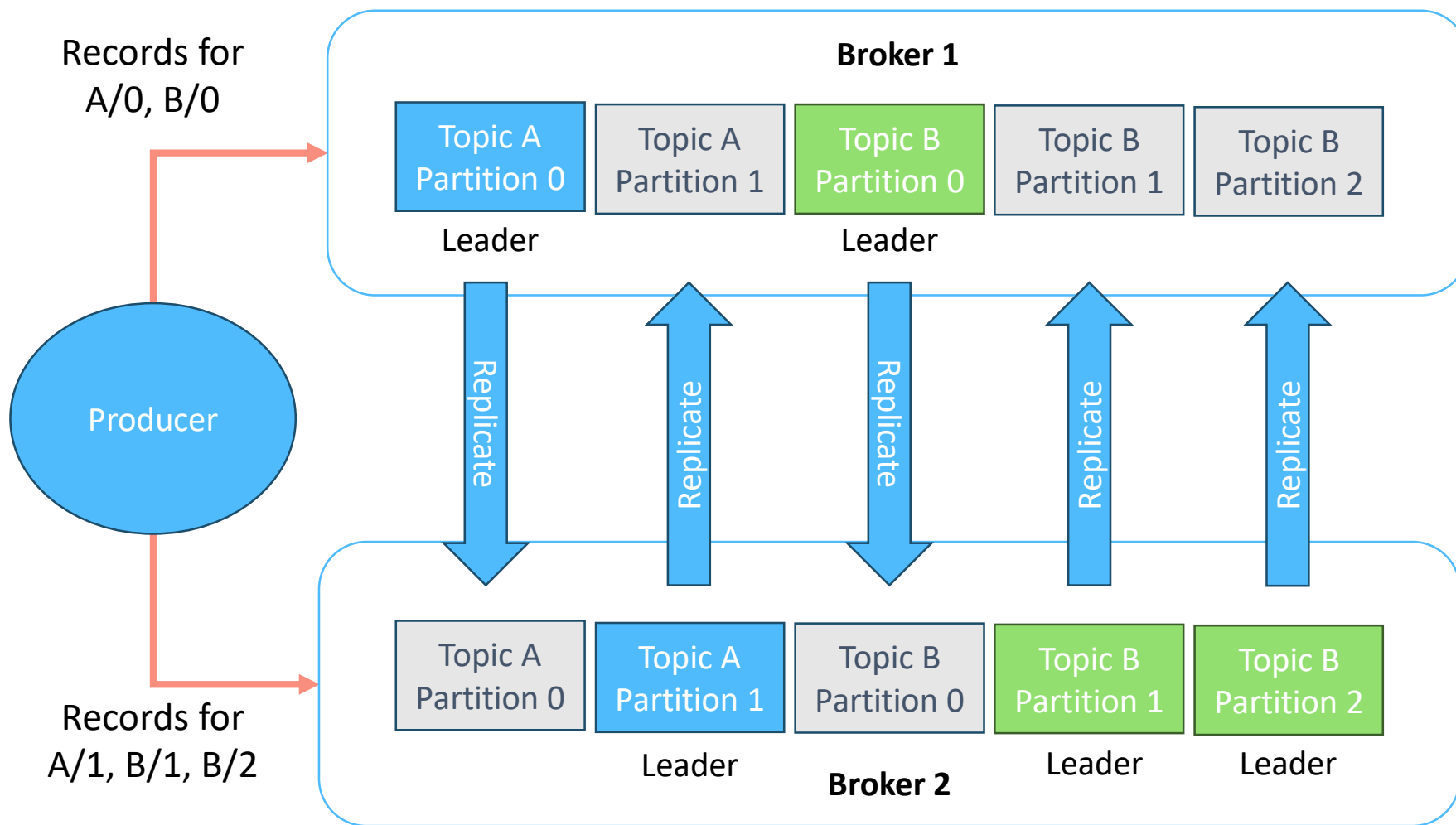
Topic B
Partition 1

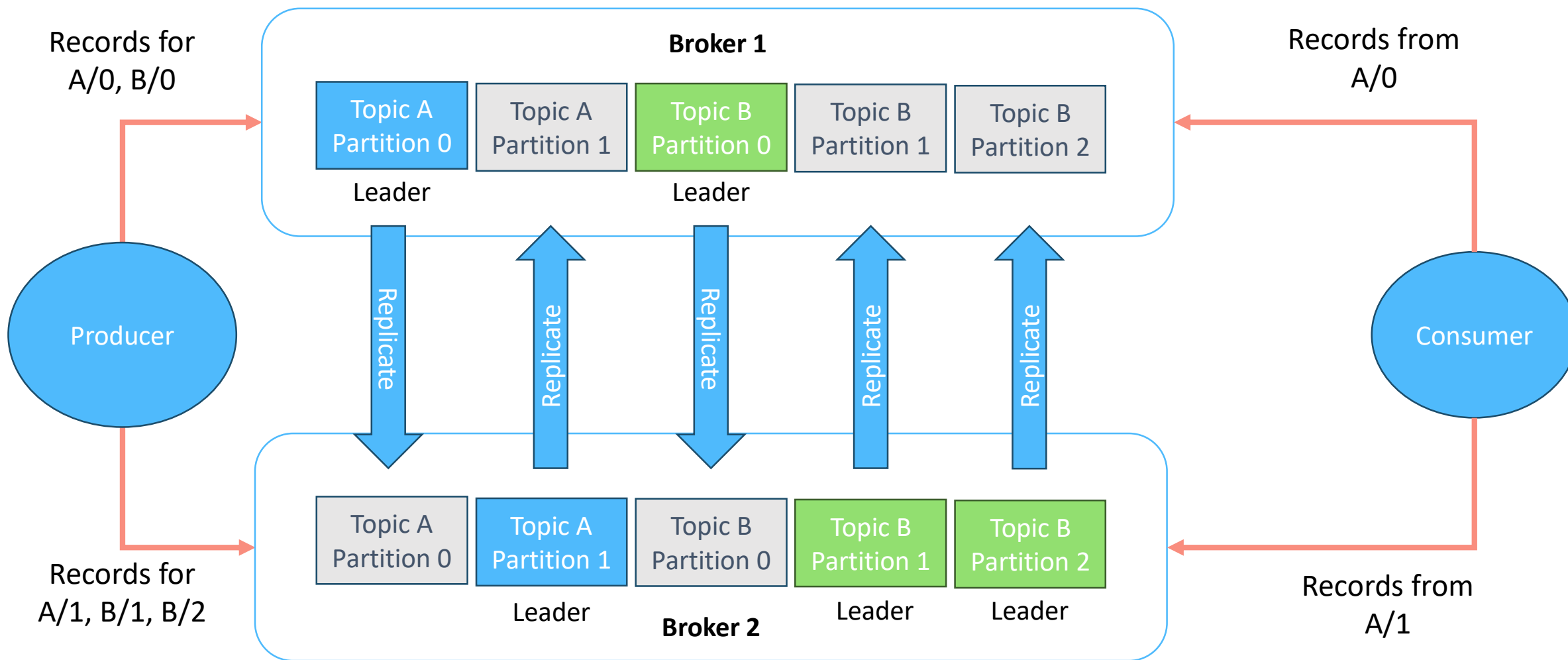
Topic B
Partition 2

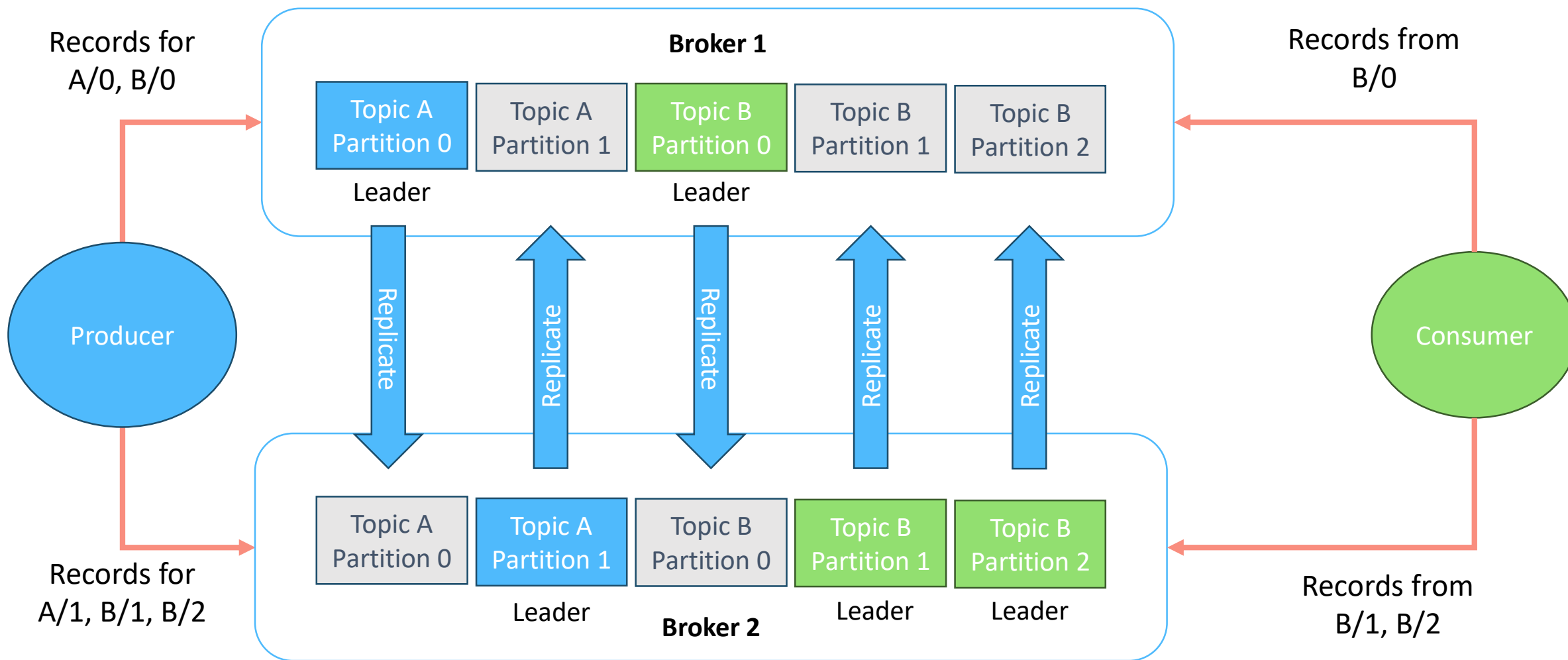
Broker 2

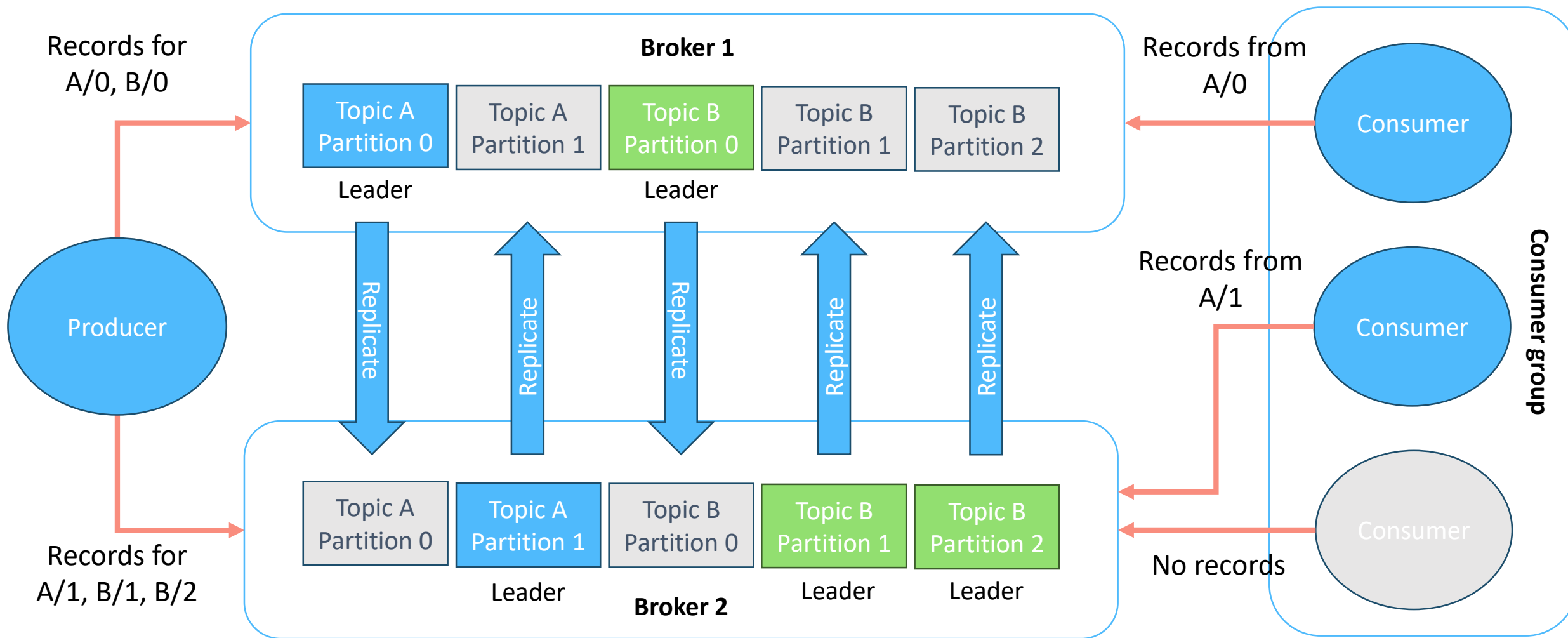


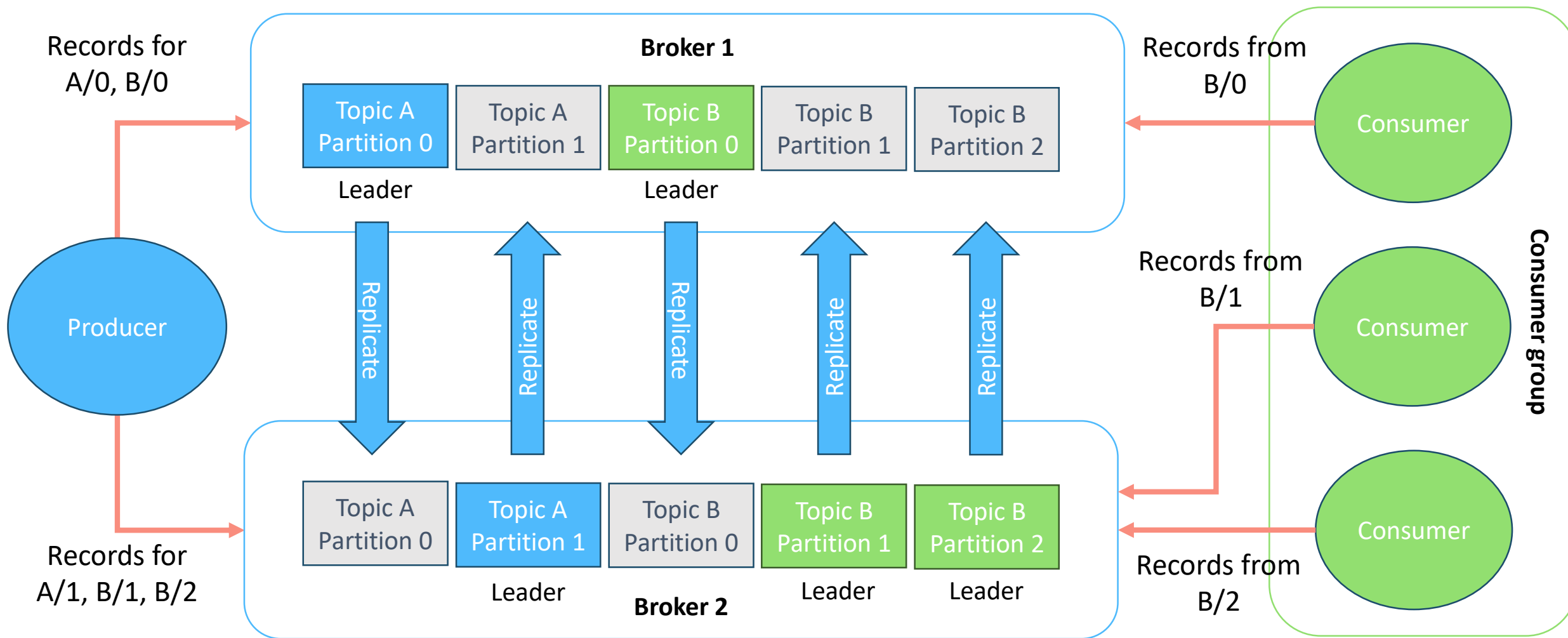


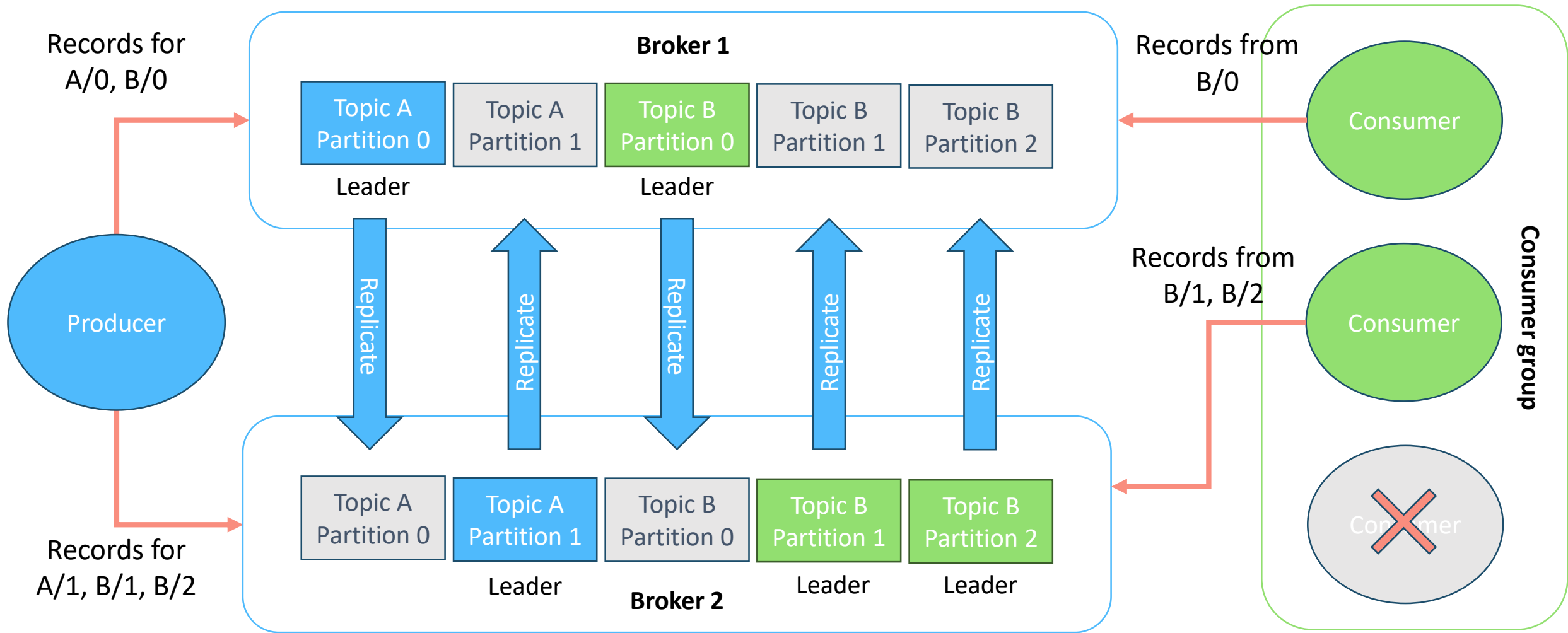












Kafka components and how they interact

- Brokers
- Topics and Partitions
- Records
- Producers
- Consumers
- ZooKeeper
- Metadata Management: brokers, topics, and partitions
- Leader Election
- Configuration Management
- Cluster Health Monitoring

Kafka components and how they interact

- Brokers
- Topics and Partitions
- Records
- Producers
- Consumers
- ZooKeeper
- KRaft (Kafka Raft)
- Integrated consensus controller
- KRaft replaces ZooKeeper
- Broker-Only Architecture
- Simplified Deployment
- Unified Configuration

Kafka components and how they interact

Components:

- Brokers
- Topics and Partitions
- Records
- Producers
- Consumers
- ZooKeeper
- KRaft (Kafka Raft)

Interactions:

- Producers write records to topics
- Topics are partitioned
- Partitions maintained by brokers
- Partitions are replicated
- Consumers, united in groups, read records from partitions

Monitoring

Documentation for Monitoring

- <https://kafka.apache.org/documentation/#monitoring>
- <https://docs.confluent.io/platform/current/kafka/monitoring.html>

Main health metrics

- ActiveControllerCount = 1
- OfflinePartitionsCount = 0
- UnderReplicatedPartitions = 0
- UncleanLeaderElectionsPerSec = 0
- UnderMinIsrPartitionCount = 0

ActiveControllerCount

- $\text{sum}(\text{ActiveControllerCount}) = 1$

For KRaft mode, the number of active controllers in the cluster.
Valid values are '0' or '1'.

For ZooKeeper, indicates whether the broker is the controller broker.

Alert if the aggregated sum across all brokers in the cluster is anything other than 1 because there should be exactly **one controller per cluster**.

OfflinePartitionsCount

The number of partitions that don't have an active leader and are therefore not writable or readable.

Alert if value is greater than 0.

UnderReplicatedPartitions

The number of under-replicated partitions

$| \text{ISR} | < | \text{current replicas} |$

Replicas that are added as part of a reassignment will not count.

Alert if the value is greater than 0.

UncleanLeaderElectionsPerSec

The unclean broker leader election rate.
Should be 0.

Unclean Leader Election (ULE)

Unclean Leader Election occurs when Kafka selects a non-ISR replica (an out-of-sync replica) as the new leader.

This happens when all ISR replicas are unavailable and the cluster is configured to allow ULE (*`unclean.leader.election.enable=true`*).

The newly promoted leader may miss the most recent acknowledged records → data loss (log truncation).

Typical triggers: quorum outages, network partitions, slow disks, GC pauses, broker crashes.

UnderMinIsrPartitionCount

The number of partitions whose ISR (in-sync replicas) count is less than *minIsr*.

These partitions will be unavailable to producers who use *acks=all*

Default Queue Metrics

- MessagesInPerSec
- ErrorsPerSec
- Log/Size

Additional Metrics

- ActiveBrokerCount
- GlobalTopicCount
- GlobalPartitionCount
- PartitionCount (*per broker*)
- LeaderCount (*per broker*)
- ConsumerLag

Scenarios for Kafka Use Cases

Big Market Players and Their Kafka Use Cases

- **LinkedIn:** Activity stream data, real-time analytics
- **Netflix:** Real-time monitoring, event processing
- **Uber:** Event sourcing, trip events
- **Spotify:** User activity processing, recommendations
- **Airbnb:** Data pipelines, real-time analytics
- **Twitter:** Data ingestion, real-time analytics
- **Goldman Sachs:** Trade processing, risk management
- **Cisco:** Network telemetry, monitoring
- **Pinterest:** Event processing, recommendation algorithms
- **PayPal:** Transaction processing, fraud detection

Real-Time Analytics

- **Use Case:** An e-commerce company wants to analyze customer behavior in real-time to personalize recommendations and improve user experience.
- **Implementation:**
 - **Data Ingestion:** Kafka is used to collect real-time data from user interactions on the website, such as page views, clicks, and purchases.
 - **Stream Processing:** Kafka Streams or KSQL is used to process and analyze the data in real-time, generating insights and updating recommendation models.
 - **Data Visualization:** Processed data is sent to a real-time dashboard or analytics platform for visualization and monitoring.

Event Sourcing

- **Use Case:** A financial services company implements event sourcing to ensure accurate and reliable transaction processing.
- **Implementation:**
 - **Event Storage:** Kafka acts as the central event store, where all events are recorded as immutable logs.
 - **State Reconstruction:** Microservices consume events to reconstruct the current state of accounts or other entities.
 - **Audit and Compliance:** Kafka's log retention ensures a complete history of all events, facilitating audits and compliance checks.

Log Aggregation

- **Use Case:** A tech company aggregates logs from various applications and services for centralized monitoring and troubleshooting.
- **Implementation:**
 - **Log Collection:** Kafka Connect is used to collect logs from multiple sources, including application servers, databases, and network devices.
 - **Centralized Storage:** Logs are stored in Kafka topics, allowing for centralized management and processing.
 - **Analysis and Monitoring:** Logs are processed and analyzed using tools like Elasticsearch and Kibana for real-time monitoring and alerting.

Metrics Collection

- **Use Case:** A cloud provider collects and processes metrics from its infrastructure to monitor performance and optimize resource usage.
- **Implementation:**
 - **Metrics Collection:** Kafka producers collect metrics from various components of the infrastructure, such as servers, containers, and applications.
 - **Stream Processing:** Kafka Streams or KSQL processes the metrics in real-time to calculate aggregates, trends, and anomalies.
 - **Visualization:** Processed metrics are sent to a monitoring system like Prometheus and visualized using Grafana.

Data Integration (ETL)

- **Use Case:** A corporation integrates data from various legacy systems into a centralized data lake for unified analysis and reporting.
- **Implementation:**
 - **Data Ingestion:** Kafka Connect is used to ingest data from legacy systems, databases, and applications into Kafka topics.
 - **Stream Processing:** Data is processed and transformed in real-time using Kafka Streams, KSQL, or Flink.
 - **Data Lake Storage:** Transformed data is stored in a data lake, such as S3/Vertica, for further analysis and reporting.

Microservices Communication

- **Use Case:** A software company adopts a microservices architecture and uses Kafka to facilitate communication between services.
- **Implementation:**
 - **Event-Driven Architecture:** Kafka topics are used to publish and subscribe to events between microservices.
 - **Service Decoupling:** Microservices communicate asynchronously through Kafka, reducing dependencies and increasing resilience.
 - **State Management:** Kafka Streams are used to manage stateful interactions and aggregate data across services.

Fraud Detection / AML

- **Use Case:** A banking institution implements a real-time fraud detection system to monitor transactions and detect fraudulent activities.
- **Implementation:**
 - **Data Collection:** Kafka collects transaction data from various channels, such as ATMs, online banking, and mobile apps.
 - **Real-Time Processing:** Stream processing processes the transaction data in real-time to detect patterns and anomalies indicative of fraud or ML.
 - **Alerting and Response:** Detected fraud events are sent to an alerting system for immediate investigation and response.

IoT Data Processing

- **Use Case:** A smart city project collects and processes data from various IoT devices, such as sensors and cameras, to improve city management.
- **Implementation:**
 - **Data Ingestion:** Kafka collects data from IoT devices deployed throughout the city.
 - **Real-Time Analytics:** Kafka Streams or KSQL processes the data in real-time to derive insights and trigger actions, such as adjusting traffic signals or monitoring air quality.
 - **Integration:** Processed data is integrated with city management systems for visualization and decision-making.

... and many more

In the Part II:

- Stream processing and EventSourcing with Kafka Streams
- Kafka Connect
- Schema Registry
- Cross-Cluster Data Replication with MirrorMaker

Homework

- Run manually configured Kafka Cluster with 3+ instances
- Use either Zookeeper or KRaft
- Create simulated stream workload (logs, payments, clicks, etc.)
- Create consumers
- Simulate outage of minority, observe availability
- Simulate outage of majority, observe unavailability

Demo